Room 1 -Version 1 — Basic Prototype

Goal of this version: Get the core logic working — detect the wire, light an LED, detect the button press. No concern for noise or stability yet.

This was our very first attempt. We set up the minimum number of pins needed and got the logic running in a tight while loop. At this stage we did not use pull-up resistors, which later turned out to be a problem.

```
from machine import Pin
import time

# Basic input/output setup
wire_input = Pin(34, Pin.IN)
button = Pin(25, Pin.IN)
led1 = Pin(12, Pin.OUT)
ledG = Pin(5, Pin.OUT)

# Simple loop - no pull-up, no multi-LED
while True:
    if wire_input.value() == 1:
        led1.on()
    if button.value() == 0:
        ledG.on()
    time.sleep(0.1)
```

Problems noticed: Without PULL_UP resistors, the input pins picked up floating noise. The wire_input would randomly trigger even with nothing connected. We also only had one LED, giving very little visual feedback to the player.

Version 2 — Intermediate Improvements

Goal of this version: Added Pin.PULL_UP to both inputs to eliminate floating signal noise. Expanded to 3 wire-trigger LEDs and added a yellow 'system on' LED.

After testing Version 1, the wire input was randomly triggering due to floating voltage on the pin. We added internal pull-up resistors, which solved this immediately. We also added more LEDs so the puzzle had a clearer visual response — three LEDs lighting up together is much more satisfying for the player than just one.

```
from machine import Pin
import time

# Added PULL_UP resistors after floating-pin issues
wire_input = Pin(34, Pin.IN, Pin.PULL_UP)
button     = Pin(25, Pin.IN, Pin.PULL_UP)

# Added more LEDs for better visual feedback
yellow_led = Pin(14, Pin.OUT)
led1       = Pin(12, Pin.OUT)
led2       = Pin(13, Pin.OUT)
ledG       = Pin(5,  Pin.OUT)

yellow_led.on()   # indicate system is ready
```

```
while True:
    if wire_input.value() == 1:
        led1.on()
        led2.on()
    if button.value() == 0:
        print('b')  # debug print
        ledG.on()
    time.sleep(0.1)
```

What improved: Stable input readings. Three LEDs gave better visual feedback. The yellow LED clearly shows the system is powered and ready. Debug print was added to confirm button presses over serial.

Final Version — Complete Room 1 System

Final state: Clean, commented, production-ready code. Added led3 (Pin 27) to the wire detection group. All pins initialized explicitly at startup.

The final version added a third wire-detection LED (led3 on pin 27) to give the full three-LED effect. We also cleaned up all the code, added section comments, and ensured every pin is initialized to a known state at startup. This prevents LEDs being stuck on from a previous run or power cycle.

```
from machine import Pin
import time

# -----------------------
# INPUTS
# -----------------------
wire_input = Pin(34, Pin.IN, Pin.PULL_UP)
button     = Pin(25, Pin.IN, Pin.PULL_UP)

# -----------------------
# LEDs
# -----------------------
yellow_led = Pin(14, Pin.OUT)  # system ready indicator
led1       = Pin(12, Pin.OUT)  # wire triggered
led2       = Pin(13, Pin.OUT)  # wire triggered
led3       = Pin(27, Pin.OUT)  # wire triggered
ledG       = Pin(5,  Pin.OUT)  # puzzle solved (green)
```

```
# -----------------------
# INITIAL STATE
# -----------------------
yellow_led.on()   # system ON indicator
led1.off()
led2.off()
led3.off()
ledG.off()        # green LED off until puzzle solved

# -----------------------
# MAIN LOOP
# -----------------------
while True:
    # Wire detection - all 3 LEDs light up together
    if wire_input.value() == 1:
        led1.on()
        led2.on()
        led3.on()
```

```
# Button press -> signal puzzle solved
if button.value() == 0:
    print('b')   # serial debug
    ledG.on()

time.sleep(0.1)
```

Room 2 & Room 3

Rooms 2 and 3 share the same code. The puzzle requires a player to scan an input (simulating an RFID / card reader), which then waits for an IR sensor to detect the player's action. A servo motor acts as a door lock that opens automatically, closes after 15 seconds, and a buzzer + LED alarm runs until the player presses a stop button.

Version 1 — Basic Prototype

Goal of this version: Prove that the scan input can be detected and trigger a buzzer. Servo and IR sensor not added yet.

We started with just the scan input and buzzer — the simplest possible version to verify our wiring and input detection were correct. The buzzer simply turned on and off on a fixed delay with no interactive element.

```
from machine import Pin, PWM
import time

# First attempt - basic scan + buzzer, no servo
led1 = Pin(18, Pin.OUT)
buzzer = Pin(5, Pin.OUT)
scan_input = Pin(34, Pin.IN)

while True:
    if scan_input.value() == 1:
        led1.on()
        buzzer.on()   # always on, no blinking
        time.sleep(1)
        buzzer.off()
```

Problems noticed: The buzzer had no blinking — it just stayed on, which was too harsh. There was no way for a player to stop the alarm. The servo and IR sensor weren't integrated yet, so the 'door open' mechanic did not exist.

Version 2 — Intermediate: Servo + IR Added

Goal of this version: Integrated the servo motor with a PWM signal. Added the IR sensor as a second trigger. Alarm now blinks properly. Auto-close added for the servo.

In this version we introduced the servo motor. Calibrating the duty cycle values took several tries — we tested values between 25 and 130 before settling on 40 (open) and 115 (closed) as the correct positions for our mounting angle. The IR sensor was added as a second step in the puzzle flow, and the alarm loop was improved to properly blink the LED and buzzer instead of just toggling once.

We also introduced a 15-second auto-close timer using time.time(), which gives players a window to step through before the door re-locks.

```
from machine import Pin, PWM
import time

# Added servo + IR sensor, blinking alarm
led1      = Pin(18, Pin.OUT)
green_led = Pin(14, Pin.OUT)
buzzer    = Pin(5, Pin.OUT)
scan_input = Pin(34, Pin.IN)
ir_sensor  = Pin(19, Pin.IN)

# Servo PWM - tuned duty cycle values by trial
servo = PWM(Pin(25), freq=50)
OPEN  = 40
CLOSE = 115
```

```
while True:
    if scan_input.value() == 1:
        led1.on()
        if ir_sensor.value() == 0:
            servo.duty(OPEN)
            # blink loop - still no stop button logic
            for _ in range(30):
                buzzer.on();  time.sleep(0.3)
                buzzer.off(); time.sleep(0.3)
            servo.duty(CLOSE)
    time.sleep(0.1)
```

Remaining issues: The stop_button pin did not have PULL_UP configured — we found it would sometimes trigger randomly. There was also no way to cleanly exit the inner while loops, which made the logic harder to trace.

Final Version — Complete Room 2 & 3 System

Final state: Full multi-step puzzle with 5 LEDs, buzzer, servo, IR sensor, scan input, and stop button. PULL_UP added to stop_button. Clean nested loop logic with clear print statements for debugging.

The final version brings all components together into a complete, production-ready system. Key changes from v2 include adding Pin.PULL_UP to the stop_button (which fixed the random triggering), expanding from 2 to 5 LEDs for richer visual feedback, clearly separating each step with comments, and ensuring the buzzer is explicitly turned off before the green LED turns on.

We also added led4 as a dedicated alarm LED that blinks in sync with the buzzer, making the alert state visually obvious to players.

```python
from machine import Pin, PWM
import time

# -----------------------
# LED SETUP
# -----------------------
led1      = Pin(18, Pin.OUT)   # blinking status LED
led2      = Pin(4,  Pin.OUT)
led3      = Pin(26, Pin.OUT)
led4      = Pin(13, Pin.OUT)   # alarm indicator
green_led = Pin(14, Pin.OUT)   # puzzle solved

# -----------------------
# BUZZER SETUP
# -----------------------
buzzer = Pin(5, Pin.OUT)
```

```python
# -----------------------
# INPUT SETUP
# -----------------------
scan_input  = Pin(34, Pin.IN)
ir_sensor   = Pin(19, Pin.IN)
stop_button = Pin(27, Pin.IN, Pin.PULL_UP)  # added PULL_UP after floating issue

# -----------------------
# SERVO SETUP
# -----------------------
servo = PWM(Pin(25), freq=50)
OPEN  = 40    # calibrated duty cycle
CLOSE = 115   # calibrated duty cycle
```

```python
# -----------------------
# INITIAL STATE
# -----------------------
led1.on()        # system ready
green_led.off()

# -----------------------
# MAIN LOOP
# -----------------------
while True:
    # STEP 1: Wait for scan input (RFID / card reader signal)
    if scan_input.value() == 1:
        led2.on()
        led3.on()
        print('Scan detected')
```

```python
    # STEP 2: Wait for IR sensor (motion / object detection)
    while True:
        if ir_sensor.value() == 0:
            print('IR detected')

            # Open servo (door unlocks)
            servo.duty(OPEN)
            start_time   = time.time()
            servo_closed = False

            # ALERT LOOP - blink LED + buzzer until stopped
            while True:
                led4.on()
                buzzer.on()
                time.sleep(0.3)
                led4.off()
                buzzer.off()
                time.sleep(0.3)
```

```python
                # Auto-close servo after 15 seconds
                if not servo_closed and (time.time() - start_time >= 15):
                    servo.duty(CLOSE)
                    servo_closed = True
                    print('Servo closed after 15 sec')

                # Stop button ends the alarm
                if stop_button.value() == 0:
                    print('Button pressed')
                    led1.off()
                    buzzer.off()
                    green_led.on()   # success indicator
                    break
            break
    time.sleep(0.1)
```